Reporte de resultados: Driver de software para un relevador

Práctica 4.1.1-A · Sistemas Programables

1. Objetivo de las pruebas

Comprobar que la clase `Relevador` controla dos actuadores con lógica opuesta (el motorreductor a través del L298N, activo en alto, y un LED conectado al revés, activo en bajo) usando el mismo código; que cada orden recibida por el Monitor Serie se traduzca al nivel de pin correcto; que la clase rechace los cambios hechos con menos de 1 s de diferencia, y que el sistema arranque siempre en el estado seguro (todo apagado).

2. Condiciones de prueba

- Arduino UNO R4 WiFi, módulo L298N y motorreductor TT alimentado por dos pilas 18650 en serie (≈ 7.4 V), con tierra común entre el portapilas, el L298N y el Arduino.
- Motor en D7 → IN1 (IN2 fijo a GND, jumpers ENA y 5V-EN puestos). LED con el ánodo a 5 V y el cátodo, a través de una resistencia de 220 Ω, a D6.
- Dos objetos de la misma clase: `motor(7, activo en alto)` y `led(6, activo en bajo)`, con un tiempo mínimo de 1000 ms entre cambios.
- Órdenes enviadas desde el Monitor Serie del Arduino IDE: M1/M0, L1/L0 y E.
- Práctica realizada en el salón de clases, en equipo.

3. Datos registrados

La tabla transcribe las respuestas visibles en la captura del Monitor Serie, en el orden en que aparecen. La columna «Orden» se deduce de cada respuesta; las dos columnas finales indican el estado que reporta el programa después de cada orden.

#	Orden	Respuesta del programa	Motor	LED
1	L1	LED ENCENDIDO	apagado	encendido
2	L0	LED APAGADO	apagado	apagado
3	M1	Motor ENCENDIDO	encendido	apagado
4	M0	Motor APAGADO	apagado	apagado
5	L1	LED ENCENDIDO	apagado	encendido
6	L0	LED APAGADO	apagado	apagado
7	M1	Motor ENCENDIDO	encendido	apagado
8	M0	Motor APAGADO	apagado	apagado
9	M1	Motor ENCENDIDO	encendido	apagado

#	Orden	Respuesta del programa	Motor	LED
10	L1	LED ENCENDIDO	encendido	encendido
11	E	Estado: Motor ENCENDIDO, LED ENCENDIDO	encendido	encendido
12	E	Estado: Motor ENCENDIDO, LED ENCENDIDO	encendido	encendido
13	L0	LED APAGADO	encendido	apagado
14	M0	Motor APAGADO	apagado	apagado
15	E	Estado: Motor APAGADO, LED APAGADO	apagado	apagado

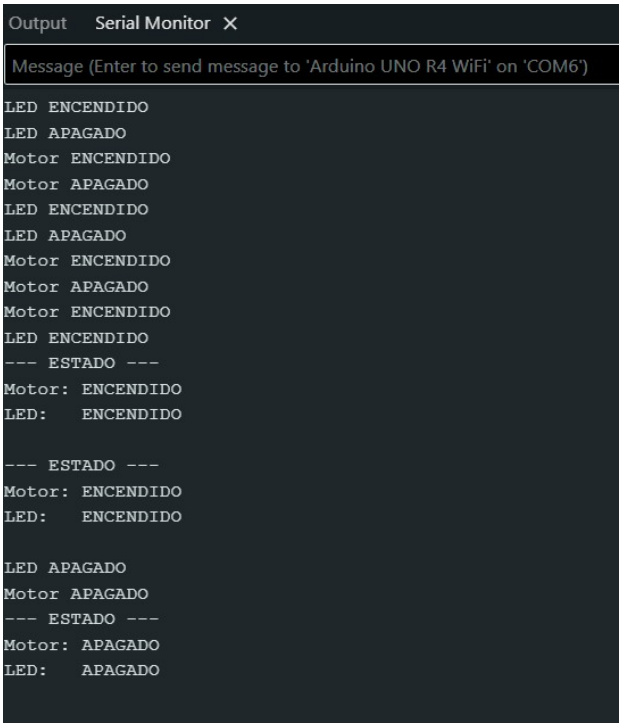


Figura 1. Captura del Monitor Serie durante la prueba.

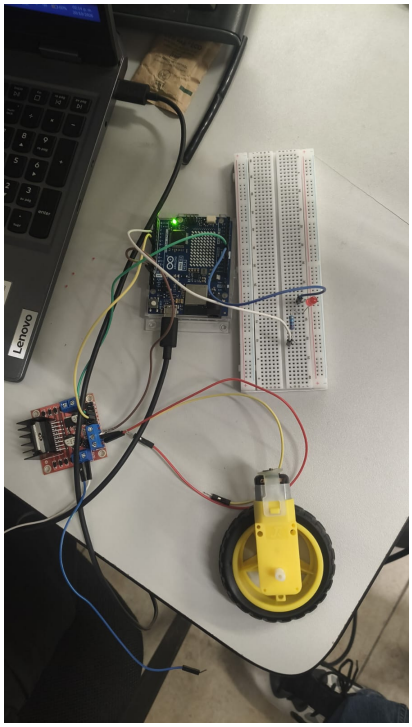


Figura 2. Montaje físico utilizado en la prueba.

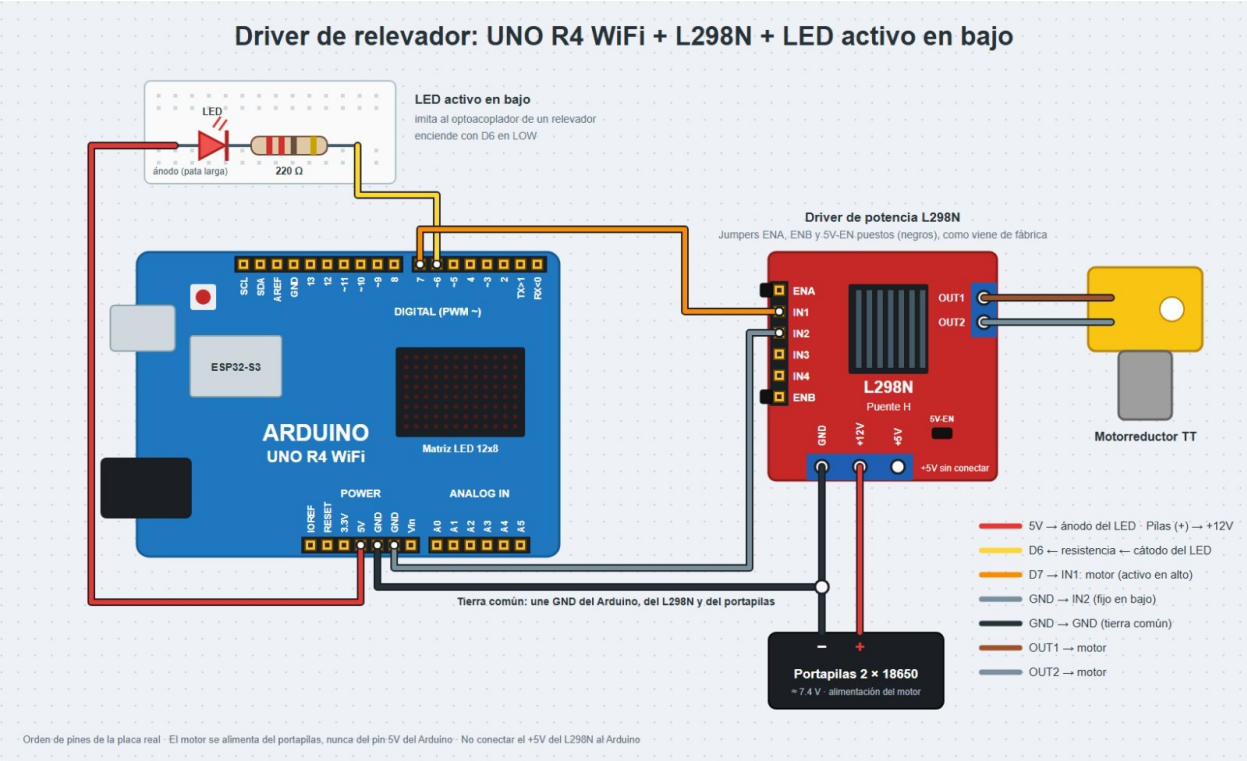
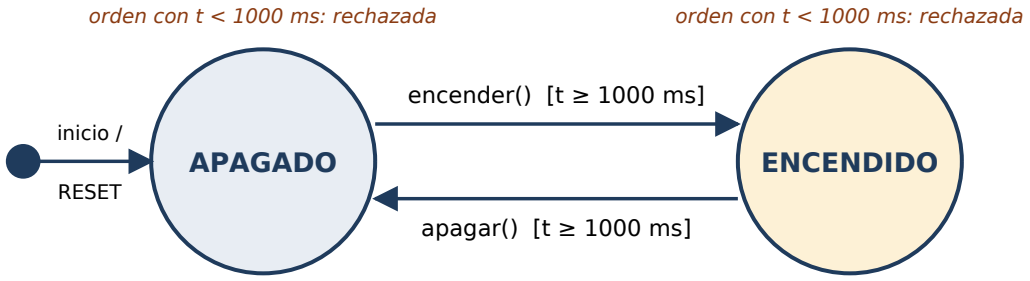


Figura 3. Diagrama de conexión: parte de señal (Arduino) y parte de potencia (portapilas y L298N).

4. Diseño y verificación

4.1 Diagrama de estados

El relevador se modeló con dos estados. Al encender o reiniciar, `iniciar()` lo deja en APAGADO. Solo se cambia de estado si ya pasó el tiempo mínimo desde el último cambio; si no, la orden se rechaza y el pin no se toca.



t = tiempo desde el último cambio, medido con millis(); el arranque cuenta como cambio

Figura 4. Diagrama de estados del relevador.

4.2 Traducción de la orden al nivel del pin

La clase escribe en el pin el nivel lógico del estado, invertido cuando el actuador es activo en bajo. Con esta regla, el mismo método produce niveles opuestos en cada actuador:

Estado pedido	Motor (D7, activo en alto)	LED (D6, activo en bajo)
ENCENDIDO	HIGH: IN1 en alto, el L298N alimenta al motor	LOW: circula corriente por el LED
APAGADO	LOW: IN1 = IN2 = bajo, el motor se detiene	HIGH: ambos extremos a 5 V, sin corriente

Las respuestas 1 a 10 confirman que esta traducción funciona: cada orden L1 encendió el LED y cada M1 encendió el motor, aunque en un caso el pin se lleva a LOW y en el otro a HIGH.

4.3 Corriente del LED

Suponiendo una caída de unos 2 V en el LED rojo: I = (5 V − 2 V) / 220 Ω ≈ 13.6 mA. Esa corriente entra al pin D6 cuando está en LOW. Es mayor que los ≈ 8 mA que la práctica indica para un pin del UNO R4; para quedar dentro de ese valor bastaría una resistencia de 390 a 470 Ω (≈ 7.7 a 6.4 mA), con el LED un poco menos brillante.

4.4 Prueba de inversión

Si el objeto del LED se creara por error como activo en alto, `iniciar()` escribiría LOW para «apagarlo», pero LOW lo enciende: el LED arrancaría encendido y se rompería el estado seguro. Después, L1 escribiría HIGH y el LED se apagaría, L0 lo encendería, y el comando E reportaría lo contrario de lo que se ve. El modelo de software dejaría de coincidir con el hardware; con un relevador real, la carga se activaría sola al encender el sistema.

5. Análisis de resultados

- **Una sola clase, dos lógicas:** el motor y el LED respondieron a sus órdenes con el mismo código; la diferencia entre activo en alto y en bajo quedó resuelta dentro de la clase.
- **Órdenes con significado:** el programa principal solo envía encender o apagar; ninguna parte del `loop()` escribe directamente en los pines.
- **Actuadores independientes:** en las líneas 10 a 13 el motor y el LED estuvieron encendidos al mismo tiempo, y apagar el LED no afectó al motor.
- **Estado consistente:** el comando E reportó en cada consulta el estado que correspondía a las órdenes anteriores (líneas 11, 12 y 15).
- **Estado final seguro:** la prueba terminó con ambos actuadores apagados.

Alcance: la captura documenta el funcionamiento normal. En ella no aparece el mensaje de orden rechazada ni el reinicio con RESET, por lo que la regla del tiempo mínimo y el arranque en estado seguro se respaldan con el diseño de la clase y no con esta captura.

6. Preguntas de reflexión

1. ¿Por qué el LED se enciende con LOW y no con HIGH? Explica el recorrido de la corriente.

El ánodo está fijo a 5 V y el cátodo va, a través de la resistencia, al pin D6. Con D6 en LOW (0 V) hay una diferencia de unos 5 V: la corriente sale del pin de 5 V, atraviesa el LED y la resistencia, y entra al pin D6, que la «hunde» hacia GND dentro del microcontrolador. Con D6 en HIGH ambos extremos quedan a 5 V, no hay diferencia de potencial y no circula corriente. Así funciona el LED del optoacoplador en los módulos de relevador comerciales.

2. ¿Qué ventaja tiene que la regla del tiempo mínimo viva dentro de la clase y no en el loop()?

Nadie puede saltársela: cualquier parte del programa que mande una orden pasa forzosamente por la regla. Cada objeto lleva su propio temporizador, el loop() queda limpio y la clase se puede reutilizar en otros proyectos con la protección incluida. Si la regla estuviera en el loop(), habría que repetirla en cada lugar que conmuta y bastaría un olvido para dañar el relevador.

3. ¿Por qué un relevador real no sirve para controlar velocidad con PWM?

Porque es mecánico: sus contactos tardan varios milisegundos en moverse y rebotan, mientras que una señal PWM conmuta cientos o miles de veces por segundo. El relevador no alcanzaría a seguir la señal, y cada conmutación produce arco eléctrico y desgaste; su vida útil se mide en cientos de miles de operaciones, que con PWM se agotarían en minutos. Para PWM se usan interruptores electrónicos como transistores, MOSFET o el puente H.

4. ¿Por qué el estado seguro es «apagado»? Da un ejemplo industrial donde convenga lo contrario (contacto NC).

Si el sistema falla, se reinicia o pierde energía, lo más seguro suele ser que nada se mueva ni se caliente: un motor que arranca solo puede lastimar a alguien o dañar la máquina. Conviene lo contrario cuando una función debe seguir activa ante una falla, por ejemplo una bomba de enfriamiento o un extractor de gases en una zona con riesgo de acumulación: se conectan al contacto NC para que, si el control se cae, sigan funcionando.

5. ¿Por qué es obligatoria la tierra común entre el portapilas y el Arduino?

Porque un voltaje siempre se mide respecto a una referencia. El L298N interpreta la señal de IN1 comparándola con su propio GND; si ese GND no está unido al del Arduino, el HIGH de D7 no tiene referencia ni camino de retorno, el nivel queda indefinido y el motor puede no responder o comportarse de forma errática.

6. Si mañana cambiaras el L298N por un módulo de relevador real, ¿qué partes del programa tendrías que modificar?

Solo la creación del objeto: el pin y, normalmente, el parámetro de activo en bajo, porque la mayoría de los módulos con optoacoplador se activan con LOW (y quizá el tiempo mínimo, según la hoja de datos del relevador). La clase y el loop() no cambian; esa es la ventaja de encapsular el dispositivo. En el hardware sí cambia el cableado: VCC, GND y la entrada IN del módulo.

7. Conclusión

Los resultados coinciden con el diseño: con una sola clase se controlaron un actuador activo en alto y otro activo en bajo, y cada orden produjo el efecto esperado sin que el programa principal tocara los pines. El comando de estado reflejó siempre lo que pasaba en el hardware y la prueba terminó en el estado seguro.

La práctica mostró que modelar el dispositivo por software, con su estado, sus órdenes y sus reglas físicas dentro de la clase, protege al hardware de errores del resto del programa y permite cambiar de actuador modificando solo la línea donde se crea el objeto.